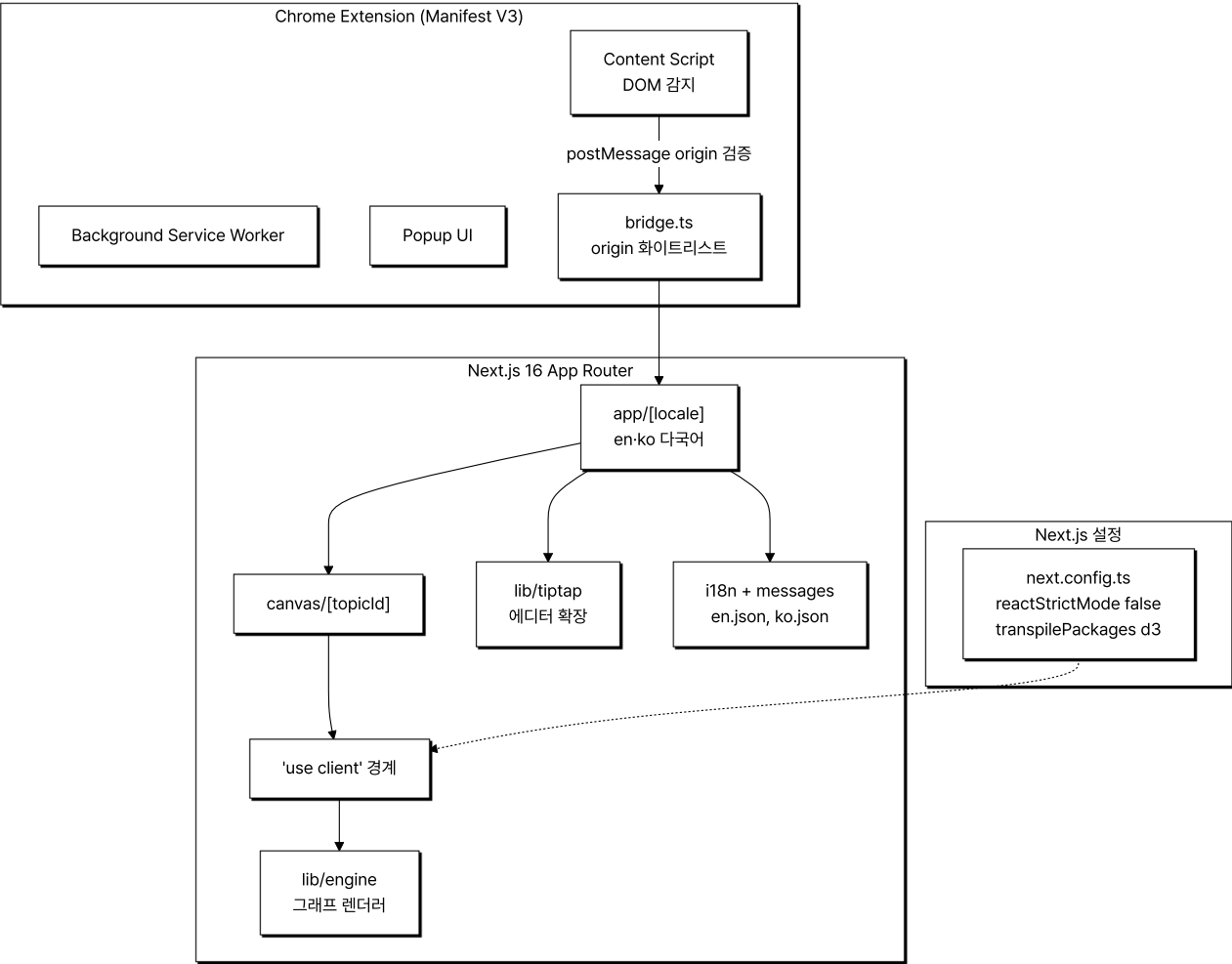


Portfolio

[MINDGRAPH] - AI 지식 캡처 & 그래프 시각화

ChatGPT·Gemini·Claude·Grok 4개 LLM 서비스의 답변을 캡처해 의미 단위로 묶고 D3.js로 시각화하는 Chrome Extension·Next.js 웹앱입니다. React 19·Next.js 16 App Router·D3·Tiptap·next-intl 다국어 라우팅을 한 코드 베이스에서 다루며, Vite 3종(Service Worker·Content Script·Bridge) 빌드 설정과 Next.js transpilePackages·StrictMode 정책 같은 빌드·번들 결정으로 Manifest V3 Extension과 Web 사이 메시지 통신·origin 검증까지 한 시스템 안에 두었습니다.

전체적인 아키텍처



- **Architecture:** Server Component 경계와 **'use client'** 레이어를 분리해 D3 같은 명령형 DOM 라이브러리를 한쪽에 격리하고, **[locale]** 라우팅 + en/ko 메시지로 다국어를 코드 베이스 안에서 다루며, Tiptap 에디터를 **lib/tiptap** 모듈에 모아 확장 관리합니다.

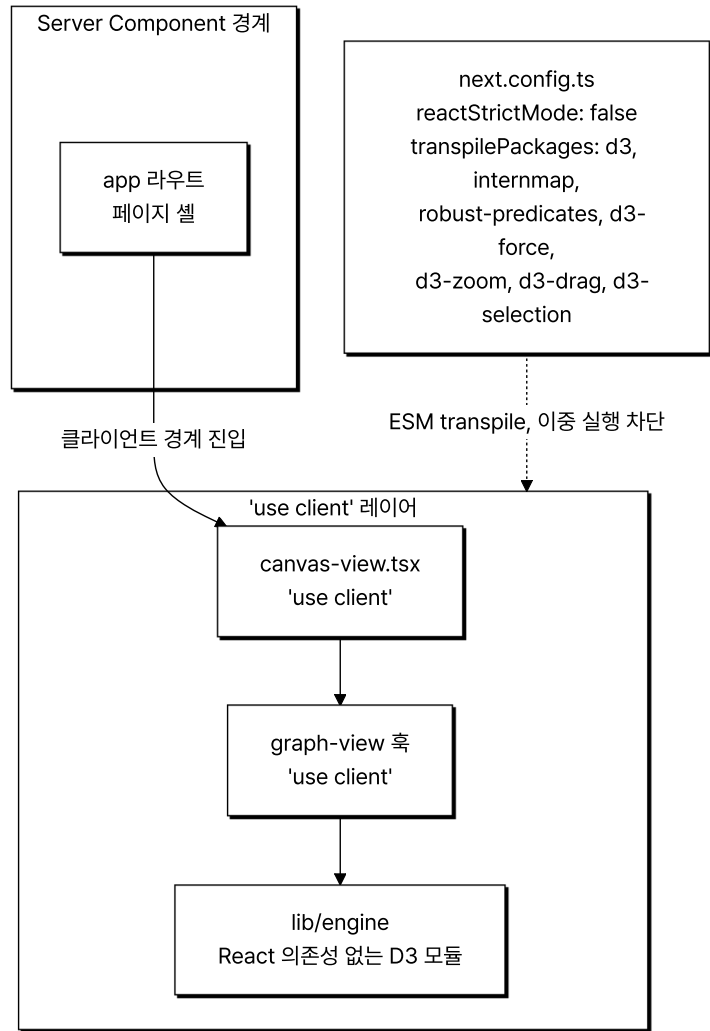
Case 1. D3.js DOM 조작과 Next.js SSR 충돌을 분리한 클라이언트 전용 그래프 엔진 격리

1. 문제 원인

- D3.js는 ESM 패키지이며 서버 환경에서 임포트하면 빌드 실패나 SSR 단계 DOM API 접근 오류가 발생했습니다.

- Next.js `reactStrictMode` 가 켜진 dev 모드에서 `useEffect` 가 두 번 실행되면서 D3 zoom이 같은 SVG에 이중 바인딩되어 훔 한 번에 두 단계가 줌되는 결함이 있었습니다.
- 그래프 엔진을 일반 모듈로 두면 Server Component에서 임포트하는 경로가 생겨 빌드 시점에 잡히지 않는 사고가 가능했습니다.

2. 해결 과정



- **엔진 모듈 격리:** `lib/engine` 그래프 렌더러를 React 의존성 없는 순수 D3 모듈로 분리하고, `'use client'` 선언된 `canvas-view.tsx` 와 `use-tree-sync` 같은 혹에서만 호출되도록 호출 경계를 단방향으로 고정했습니다.
- **reactStrictMode 비활성화:** `next.config.ts` 의 `reactStrictMode: false` 로 dev 모드 이중 실행을 차단해 D3 zoom 이중 바인딩을 막았고, 의도(**StrictMode 비활성화: D3 렌더러 이중 실행 방지**)를 주석에 함께 기록했습니다.
- **D3 ESM transpile:** `transpilePackages: ['d3', 'internmap', 'robust-predicates', 'd3-force', 'd3-zoom', 'd3-drag', 'd3-selection']` 를 설정해 D3 계열 ESM 패키지가 서버 빌드 경로에 들어와도 Next.js가 transpile하도록 했습니다.

3. 결과

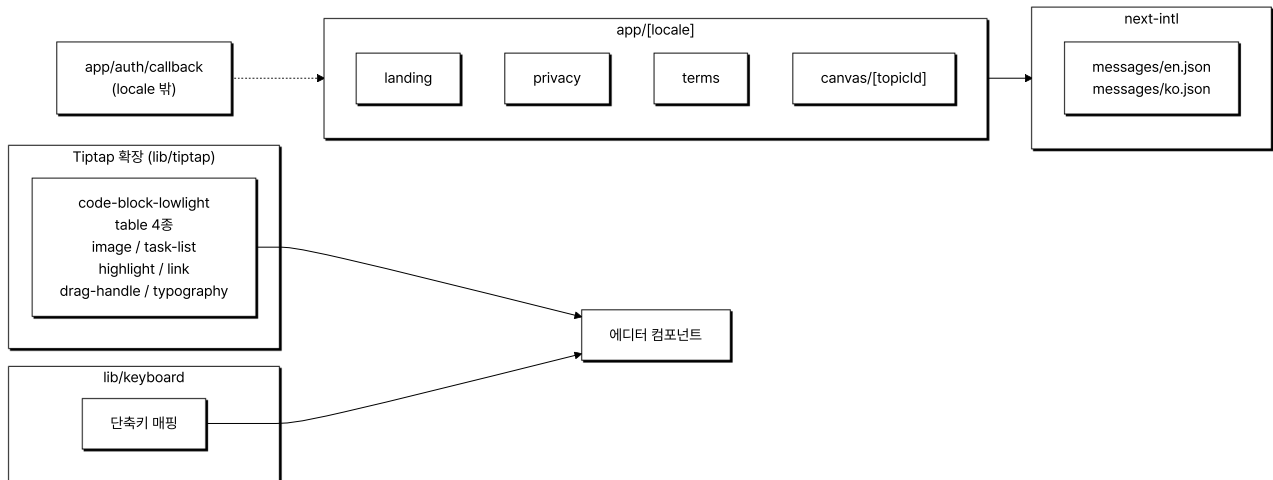
- **성과:** D3 zoom 이중 바인딩이 사라져 훔 한 번에 한 단계만 줌되도록 정상화했고, 그래프 엔진이 Server Component에 잘못 임포트되어도 영향 범위가 클라이언트 경계 안으로 한정되었습니다.
- **배운 점:** D3처럼 DOM에 직접 접근하는 라이브러리를 Next.js App Router에서 다룰 때 `'use client'` 선언 위치·ESM transpile 설정·StrictMode 정책 세 가지를 함께 맞춰, zoom 이중 바인딩 없이 렌더링되도록 했습니다.

Case 2. Tiptap 에디터 확장 모듈화와 next-intl 다국어 라우팅 단일화

1. 문제 원인

- 노드 본문 편집에 일반 contenteditable은 IME·붙여넣기·키보드 단축 처리가 약했고, Tiptap 기본 확장만으로는 LLM 답변에 자주 등장하는 코드 블록·표·이미지·체크리스트를 모두 다룰 수 없었습니다.
- 확장과 단축키가 컴포넌트 안에 분산되면 새 확장 추가나 통합 단축 정책 변경 비용이 누적됐습니다.
- 한국어·영어 양쪽 사용자를 받기 위해 메타데이터·랜딩·약관·인증 화면을 다국어로 제공해야 했고, 메시지를 컴포넌트에 하드코딩하면 번역 검수 시 코드 베이스 전체를 뒤져야 했습니다.

2. 해결 과정



- **Tiptap 확장 모듈화**: **lib/tiptap** 에 **code-block-lowlight** · **table 4종** · **image** · **task-list** · **highlight** · **link** · **drag-handle** · **typography** 확장을 모아 두고 에디터 컴포넌트에서 import해 사용했습니다.
- **단축키 분리**: **lib/keyboard** 에 단축키 매핑을 모아 에디터·캔버스·검색이 같은 단축 정책을 공유하도록 했습니다.
- **next-intl 라우트 그룹**: **app/[locale]** 동적 세그먼트 아래 **landing** · **privacy** · **terms** · **canvas/[topicId]** 를 두어 모든 사용자 화면이 locale을 거치고 메시지는 **messages/{en,ko}.json** 두 JSON에서 끝나도록 했습니다.
- **OAuth 콜백 분리**: 콜백은 **app/auth/callback** 을 별도 뒤 locale 변경 영향을 받지 않게 해 provider 등록 URL을 ko/en별로 따로 관리하지 않도록 했습니다.

3. 결과

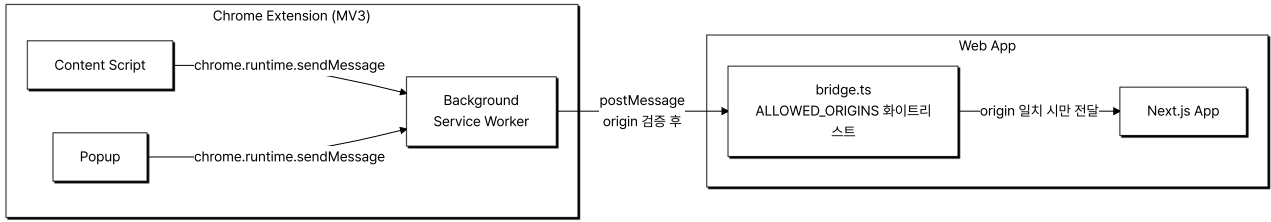
- **성과**: 새 Tiptap 확장 추가가 **lib/tiptap** 에 의존성 등록 한 단계로 끝나고, 번역 검수는 두 JSON 파일에서 마무리되며, OAuth 콜백은 다국어 라우트와 독립적으로 동작합니다.
- **배운 점**: 확장·단축키·메시지·인증 콜백 네 가지를 각각의 단일 진입점으로 모아 새 기능 추가 시 손대야 할 파일을 한 곳으로 좁혔습니다.

Case 3. Manifest V3 Extension과 Web 사이 메시지 통신·origin 검증

1. 문제 원인

- Chrome Extension MV3에서 Web 도메인으로 인증 토큰을 넘기는 표준 경로가 없어, postMessage 기반 자체 브리지를 만들어야 했습니다.
- postMessage는 origin을 명시 검증하지 않으면 외부 페이지가 임의로 토큰을 가져갈 수 있는 보안 결함이 생깁니다.
- MV3는 **eval()** 금지·외부 스크립트 동적 주입 금지 같은 CSP 제약이 있어 일반 웹앱 통신 패턴을 그대로 쓸 수 없습니다.

2. 해결 과정



- **bridge.ts 화이트리스트**: 웹앱 쪽 **bridge.ts** 가 **ALLOWED_ORIGINS** 상수에 허용 origin 목록을 두고, postMessage 수신 시 **event.origin** 이 화이트리스트에 있을 때만 처리합니다(**SECURITY_POLICY.md §8** 강제).
- **chrome.runtime.sendMessage**: Extension 내부 통신은 **eval** 없이 안전한 **chrome.runtime.sendMessage** 채널만 사용합니다.
- **3-vite 빌드 분리**: **vite.config.extension.ts** · **vite.config.content.ts** · **vite.config.bridge.ts** 3개 vite 설정으로 Service Worker·Content Script·인증 bridge 번들 요구사항을 분리해 각 환경 보안 제약을 따로 다룹니다.
- **인증 토큰 한 방향**: Extension에서 Web 방향으로만 토큰이 흐르고 역방향은 없게 해 토큰 노출 표면을 최소화했습니다.

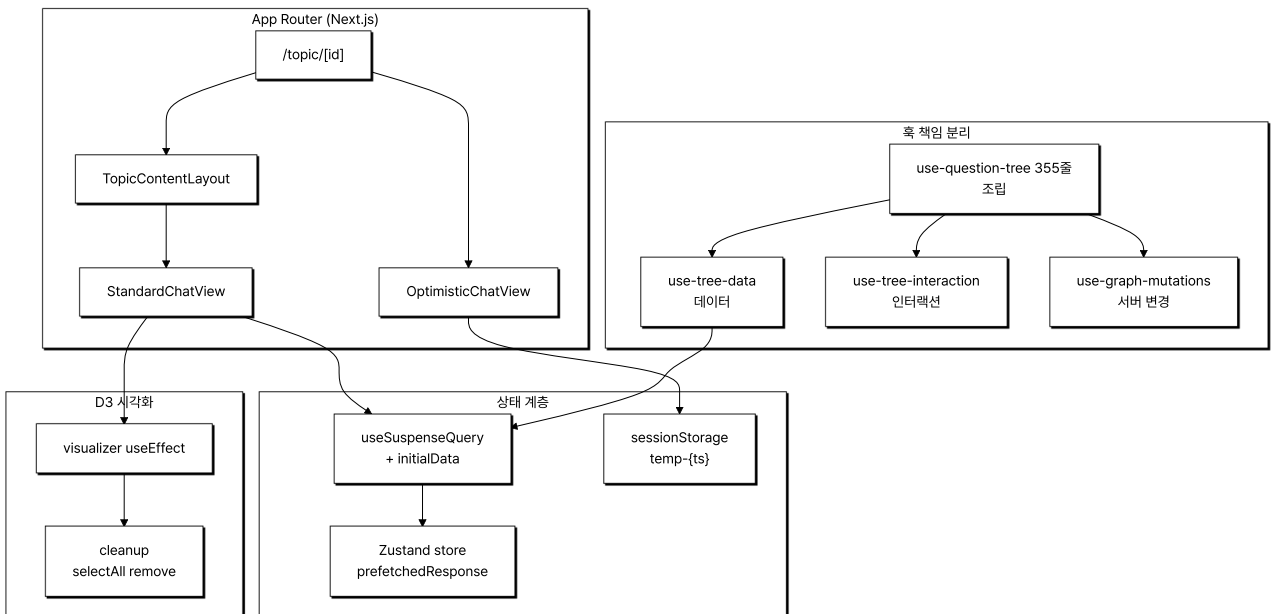
3. 결과

- **성과**: 외부 페이지가 임의로 mindgraph 인증 토큰을 가져가는 경로를 origin 화이트리스트로 차단했고, MV3 CSP 제약 안에서 Extension과 Web 사이 메시지 통신을 한 방향 흐름으로 정리했습니다.
- **배운 점**: ALLOWED_ORIGINS 화이트리스트를 bridge.ts에 두고 Service Worker·Content Script·bridge 번들을 3개 vite 설정으로 분리해, MV3 CSP 위반 없이 인증 토큰이 화이트리스트 origin으로만 전달되도록 했습니다.

[CHATGRAPH] - AI 대화 시각화 플랫폼

기존 선형 채팅 UI에서 한 갈래 흐름만 살아남고 도중에 떠올린 분기 질문·맥락이 휘발되어 사용자가 이전 맥락을 다시 짜맞춰야 하는 비용이 컸습니다. chatGraph는 AI와의 대화를 꼬리물기 형태의 그래프로 시각화해 분기와 깊이를 보존하는 웹 서비스입니다. 본인은 4인 팀의 FE 한 명으로 참여해 React 라이프사이클 통합·Suspense 경계 설계·D3 cleanup 패턴을 담당했습니다.

전체적인 아키텍처



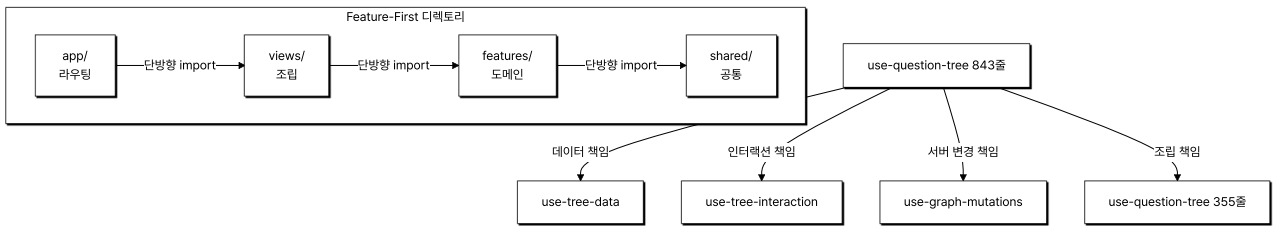
- **Architecture:** Feature-First 디렉토리(`app` · `features` · `views` · `shared`) 위에 혹 책임을 4개로 분리하고, 라우트별로 Suspense 경계와 Optimistic 경로를 나눠 D3 시각화를 React useEffect cleanup으로 안전하게 통합한 구조.

Case 1. 거대 혹 책임 분리와 Feature-First 디렉토리 도입 (843줄에서 355줄로 축소)

1. 문제 원인

- 트리 상태·인터랙션·서버 변경을 모두 끌어안은 `use-question-tree` 혹이 843줄까지 비대해져 변경 시 영향 범위 파악이 어려웠습니다.
- 페이지·기능·공통 코드 경계가 모호해 컴포넌트끼리 순환 import가 반복 발생했고, 새 기능 추가 시 어디에 코드를 두어야 하는지 팀 합의가 매번 필요했습니다.

2. 해결 과정



- **4-책임 혹 분리:** 단일 혹이 들고 있던 책임을 트리 데이터(`use-tree-data`)·인터랙션 상태(`use-tree-interaction`)·서버 변경(`use-graph-mutations`)으로 분리하고 최상위에서 합성하는 형태로 정리했습니다.
- **단방향 import 정비:** `app` 은 라우팅· `features` 는 도메인· `views` 는 조립· `shared` 는 공통이라는 네 책임으로 디렉토리를 재정의하고 import 방향을 단방향으로 고정해 순환 참조를 차단했습니다.
- **점진적 리팩토링:** 본인 단독 커밋 3건에 걸친 누적 리팩토링으로 진행해 단일 거대 PR 위험을 분산했습니다.

3. 결과

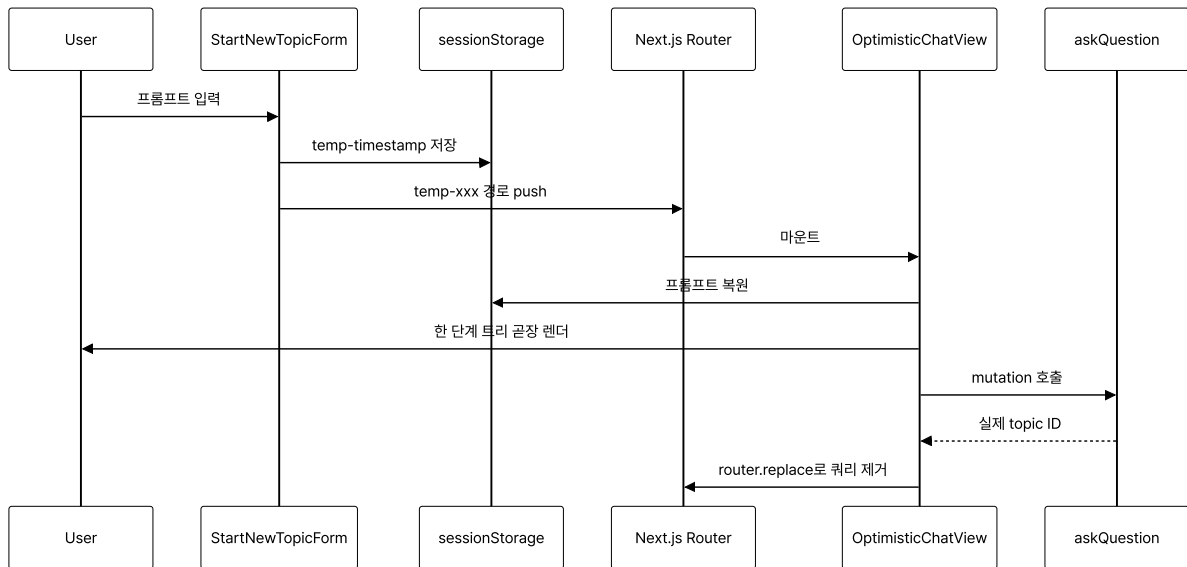
- **성과:** 핵심 혹을 843줄에서 355줄로, 인접 페이지 컴포넌트를 186줄에서 59줄로 축소하고 디렉토리 책임이 명확해져 팀 머지 충돌 빈도가 줄었습니다.
- **배운 점:** `app`·`views`·`features`·`shared` 단방향 import로 고정한 뒤 새 기능 추가 시 코드 위치 합의에 드는 시간이 줄고 머지 충돌 빈도가 감소했습니다.

Case 2. Optimistic 라우팅으로 첫 입력과 첫 화면 사이 단절 제거

1. 문제 원인

- 첫 질문 입력 시 LLM 응답이 도착할 때까지 화면이 멈춘 듯한 경험이 사용자 물입을 끊었습니다.
- 토스트·스피너 방식은 생성될 URL을 미리 보여주지 못해 뒤로가기·새로고침·공유 동작이 어색했습니다.

2. 해결 과정



- **임시 ID + sessionStorage:** `StartNewTopicForm` 에서 `temp-{Date.now()}` 임시 ID를 만들어 `sessionStorage`에 프롬프트를 보관하고(`use-start-new-topic.ts`) 임시 경로로 push해 사용자 입력이 그려진 화면을 보여줍니다.
- **가짜 트리 렌더:** 이동한 경로에서 `useOptimisticTopicData` 훅이 `sessionStorage` 데이터를 복원해 한 단계짜리 트리를 렌더링하면서 동시에 실제 mutation을 실행합니다.
- **router.replace 정책:** mutation 성공 시 Zustand 스토어에 응답을 채우고 `router.replace` 로 optimistic 쿼리 파라미터를 제거해 실제 topic ID 경로로 자연스럽게 전환합니다.

3. 결과

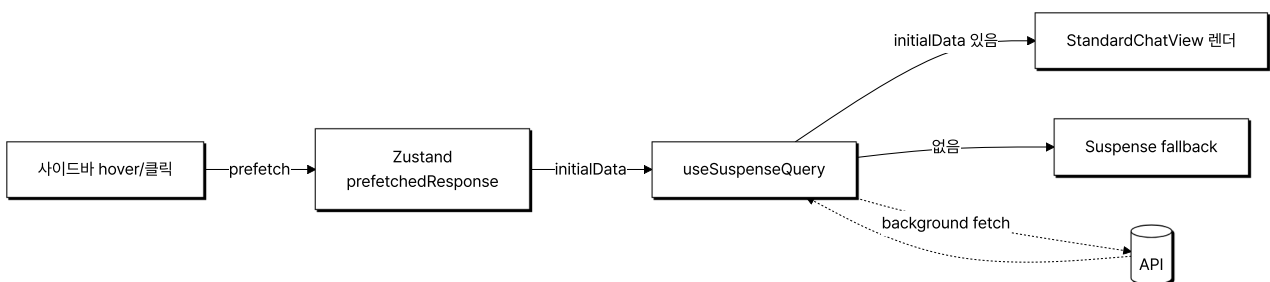
- **성과:** 첫 입력과 첫 화면 사이의 단절을 없애 가장 큰 이탈 지점을 막았고, 새로고침·공유 링크도 정상 동작합니다.
- **배운 점:** 임시 ID 라우팅·sessionStorage 복원·mutation 성공 후 router.replace를 한 흐름으로 묶어 첫 입력 직후 빈 화면 노출 없이 채팅 화면이 그려졌습니다.

Case 3. useSuspenseQuery initialData + Zustand 프리패치로 fallback 우회

1. 문제 원인

- 사이드바에서 다른 토픽 클릭 시 `useQuery`가 새로 호출되며 페이지가 잠시 빈 상태로 보였고, App Router Suspense fallback이 노출되며 깜빡임이 발생했습니다.
- 데이터 양이 많은 토픽일수록 체감 지연이 커졌습니다.

2. 해결 과정



- **사이드바 prefetch:** 사이드바에서 토픽 응답을 미리 받아 Zustand 스토어의 `prefetchedResponse` 슬롯에 저장합니다.

- **initialData 분기:** 페이지 진입 시 `useSuspenseQuery` 의 `initialData`에 동일 `topicId`의 프리패치 응답이 있으면 그대로 사용(`use-topic-data.ts`)하고, 없을 때만 `Suspense fallback`을 거치도록 분기했습니다.
- **트리 변환:** `transformApiDataToViewData` 어댑터가 백엔드 응답을 트리 형태로 변환해 주입(`use-tree-data.ts`)합니다.
- **잔여 정리:** 사용한 프리패치 데이터는 `effect cleanup`에서 정리해 다음 토픽 전환 시 잔여 데이터가 섞이지 않도록 했습니다.

3. 결과

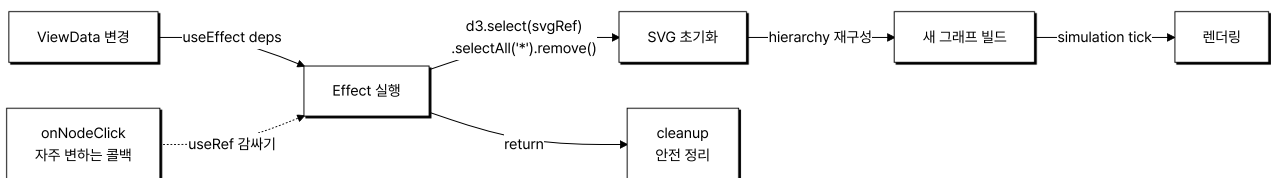
- **성과:** 사이드바 기반 라우트 전환에서 빈 화면 노출 없이 트리가 그려지는 흐름을 확보했습니다.
- **배운 점:** 사이드바 `prefetch`를 `Zustand prefetchedResponse`에 적재하고 `useSuspenseQuery`의 `initialData`로 주입해 사이드바 토픽 전환 시 `Suspense fallback` 노출 케이스를 `prefetch` 미적재 경로로만 한정했습니다.

Case 4. D3 + React useEffect cleanup 패턴으로 시각화 라이프사이클 안전화

1. 문제 원인

- 팀원이 도입한 `D3 Force Simulation`은 `SVG DOM`을 직접 변형해서 `React`가 같은 `SVG`를 다시 그릴 때 잔여 노드·이벤트 리스너가 남아 시각적 결함이 발생했습니다.
- 토픽이 바뀌거나 트리가 갱신될 때마다 이전 시뮬레이션 상태가 새 데이터 위에 덧씌워졌습니다.

2. 해결 과정



- **SVG 초기화 패턴:** `visualizer` 컴포넌트 `useEffect`에서 `d3.select(svgRef).selectAll("*").remove()` 로 매 갱신마다 `SVG` 하위 트리를 초기화한 뒤 `hierarchy-simulation`을 다시 구성하는 패턴을 정리했습니다.
- **deps 최소화:** `onNodeClick` 같이 자주 변하는 콜백은 `ref`로 감싸 `effect` 의존성에서 분리하고, `currentPath` 등 시각적 강조에 영향을 주는 값만 `deps`에 포함해 불필요한 재구성을 막았습니다.
- **팀 역할 분담:** 색상 알고리즘·호버 `desaturate`는 팀원 코드 유지, 본인은 통합 지점인 라이프사이클·`cleanup` 흐름에 집중했습니다.

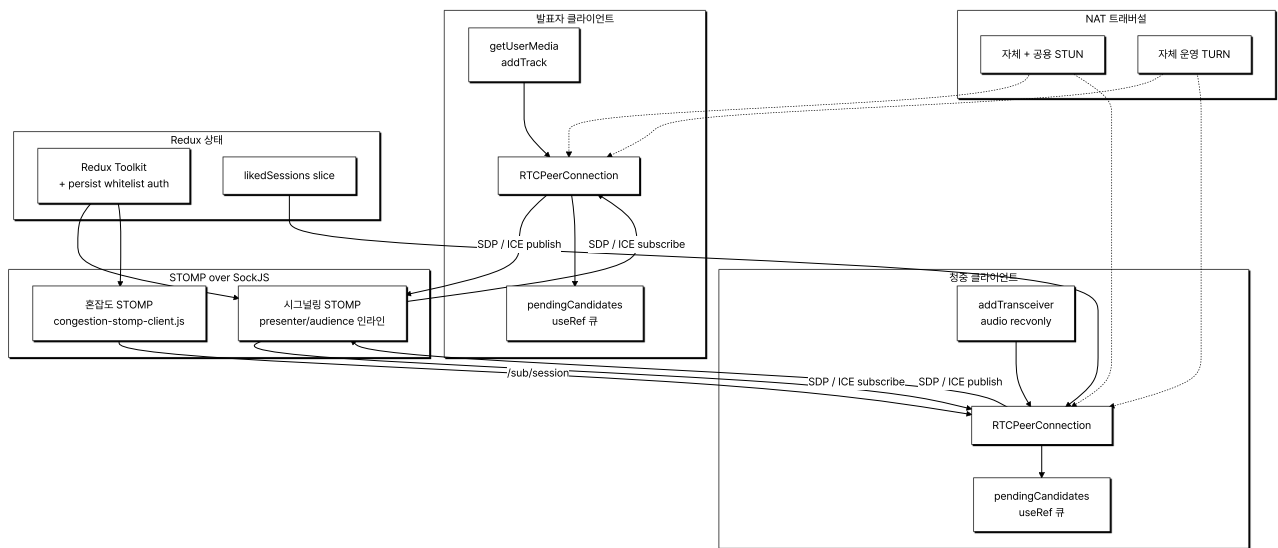
3. 결과

- **성과:** 토픽이 바뀌거나 트리가 갱신될 때 `SVG`가 정상 재구성되어 잔여 노드·중복 이벤트 시각적 결함이 사라졌습니다.
- **배운 점:** `visualizer` `useEffect`의 `cleanup`에서 `selectAll('*').remove()`로 `SVG`를 초기화한 뒤 `hierarchy`를 재구성하도록 분리해 토픽 전환 시 잔여 노드·중복 이벤트가 사라졌습니다.

[FIT] - WEBRTC 시그널링을 직접 구현한 사일런트 컨퍼런스 플랫폼

행사장에서 다중 채널 발표를 무선 헤드폰으로 골라 듣는 사일런트 컨퍼런스의 온라인·오프라인 하이브리드 중계 플랫폼입니다. 본인은 FE 3명 중 한 명으로 참여해 `RTCPeerConnection` API 직접 사용·`STOMP` 클라이언트 설계·`React useRef` 기반 이벤트 큐잉을 담당했습니다.

전체적인 아키텍처



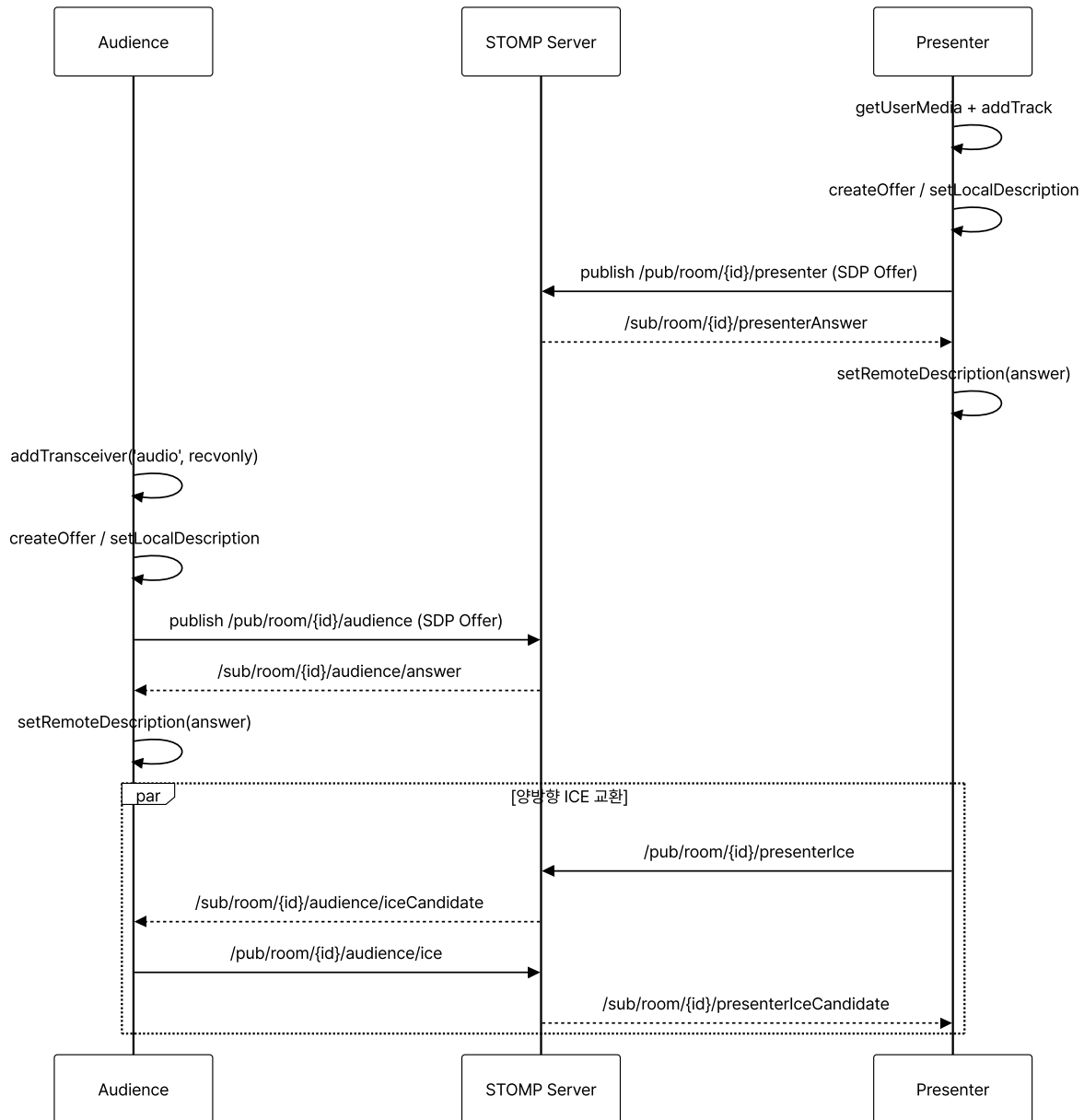
- **Architecture:** Vite + React 환경에서 `RTCPeerConnection` API를 라이브러리 래퍼 없이 직접 사용. 채팅 STOMP 클라이언트와 혼잡도 전용 STOMP 클라이언트를 이중화하고, React `useRef` 기반 ICE 큐로 비동기 메시지 순서 의존성을 클라이언트 상태 머신에서 직접 다뤘습니다.

Case 1. STOMP 위 WebRTC 시그널링 직접 구현 (라이브러리 래퍼 없음)

1. 문제 원인

- 사일런트 컨퍼런스 청취 경험은 발표자 입모양과 오디오 싱크가 어긋나면 무너지므로, HLS/RTMP 같은 수 초 단위 버퍼링 스택은 부적합하고 수백 밀리초 RTT를 확보하는 WebRTC가 필요했습니다.
- 사내 인프라에 별도 WebRTC SDK가 없어 시그널링 채널을 직접 정의해야 했고, 발표자·청중 양측 `PeerConnection` 라이프사이클을 React 컴포넌트 안에서 직접 관리해야 했습니다.

2. 해결 과정



- 발표자 **PeerConnection**: **getUserMedia** 로 마이크 스트림을 받아 **addTrack** 으로 PeerConnection에 부착하고, **createOffer / setLocalDescription** 으로 만든 SDP를 STOMP **/pub/room/{id}/presenter** 토픽에 publish합니다.
- 청중 **recvonly** 트랜시버: 음성 송출이 필요하지 않으므로 **pc.addTransceiver('audio', { direction: 'recvonly' })** 로 수신 전용 트랜시버를 추가해 별도 PeerConnection을 만들어 SDP 협상 비용을 최소화했습니다.
- ICE 후보 양방향 **publish**: **onicecandidate** 콜백에서 생성되는 ICE 후보를 발표자/청중 각각 **/pub/room/{id}/presenterIce** · **/pub/room/{id}/audience/ice** 토픽으로 양방향 publish합니다.
- NAT 트래버설: 자체 운영 TURN 서버와 자체 STUN·공용 STUN을 **iceServers** 배열에 등록해 행사장 NAT 환경에서도 연결률을 확보했습니다.

3. 결과

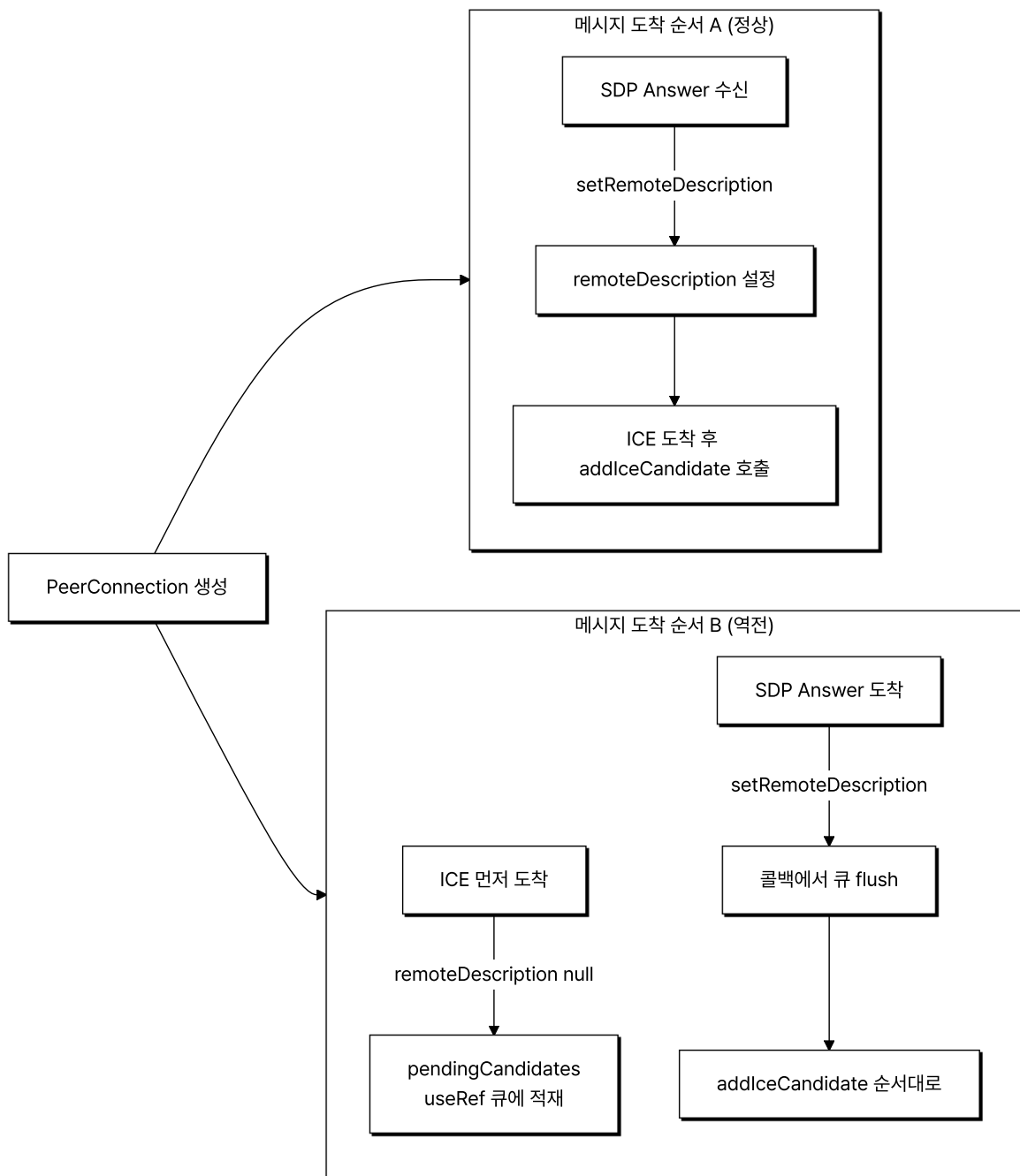
- **성과**: 발표자·청중이 동일한 STOMP 채널 위에서 SDP와 ICE를 교환하는 1:N 오디오 송수신 구성을 라이브러리 래퍼 없이 완성했습니다.
- **배운 점**: 라이브러리 래퍼 없이 createOffer·setLocalDescription·addIceCandidate를 직접 호출하면서 SDP·ICE 도착 순서를 컴포넌트 상태로 관리하는 흐름을 정리했습니다.

Case 2. React useRef 기반 ICE 큐로 비동기 메시지 순서 의존성 처리

1. 문제 원인

- 시그널링 초기 구현에서 SDP Answer가 도착하기 전에 ICE 후보가 먼저 도착하면 `setRemoteDescription` 이 호출되지 않은 `PeerConnection`에 `addIceCandidate` 를 시도하면서 예외가 발생했습니다.
- 후보 한 번 유실은 NAT 트래버설 실패로 이어져 미디어 트랙이 붙지 않는 무음 상태가 재현됐고, 실패 시 복구 경로가 없어 새로고침해야 했습니다.
- 원인은 STOMP `presenterAnswer / audience/answer` 와 `presenterIceCandidate / audience/iceCandidate` 메시지의 도착 순서가 정렬되지 않는다는 점에 있었습니다.

2. 해결 과정



- **useRef 큐:** 발표자·청중 컴포넌트 양쪽에 **pendingCandidates** 라는 **useRef** 큐를 두고, ICE 메시지 수신 시 **pcRef.current.remoteDescription** 이 비어 있으면 큐에 적재합니다.
- **flush 시점:** SDP Answer 수신 시점의 **setRemoteDescription** 콜백 안에서 큐에 쌓인 후보를 순서대로 **addIceCandidate** 로 flush 하고 큐를 비웁니다.
- **컴포넌트 ref 선택 이유:** PeerConnection이 컴포넌트 단위 라이프사이클을 가지기 때문에 useRef로 두었고, 전역 스토어에 두면 라우팅 전환·언마운트 시 잔여 후보가 다음 세션으로 누수될 위험이 있었습니다.

3. 결과

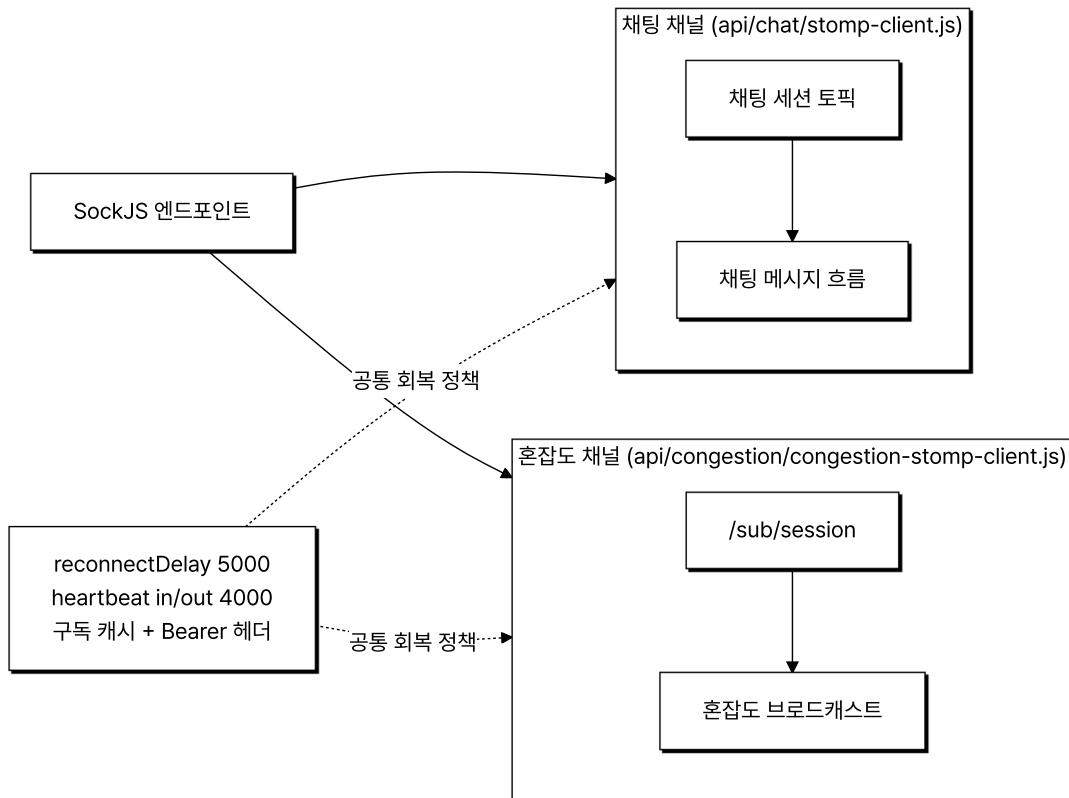
- **성과:** SDP-ICE 도착 순서 역전으로 인한 초기 연결 실패가 사라지고 PeerConnection이 ICE Connected 상태에 도달하는 동작이 일관되었습니다.
- **배운 점:** pendingCandidates useRef 큐로 SDP Answer 도착 전 ICE 후보를 보관한 뒤 setRemoteDescription 직후 flush하도록 정렬해 ICE Connected 상태 도달이 일관됐습니다.

Case 3. 채팅·혼잡도 채널의 STOMP 회복 정책과 채널 이중화

1. 문제 원인

- 행사장 Wi-Fi·모바일 망이 일시적으로 끊기는 일이 잦았고 기본 STOMP 설정의 끊김 감지가 늦어 좀비 커넥션이 유지되며 수동 재연결이 필요해, 실시간 채팅과 혼잡도 표시가 새로고침 없이는 복구되지 않았습니다.
- 채팅 STOMP 하나에 혼잡도 토픽을 묶으면 채팅 메시지가 물리는 구간에서 혼잡도 수신이 밀리거나 반대로 혼잡도 브로드캐스트가 채팅 채널을 점유할 위험이 있었습니다.

2. 해결 과정



- **회복 정책 공통화:** 채팅(**api/chat/stomp-client.js**)·혼잡도(**api/congestion/congestion-stomp-client.js**) 두 STOMP 클라이언트에 **reconnectDelay: 5000** 과 **heartbeatIncoming/Outgoing: 4000** 을 동일하게 적용해 4초 간격 양방향 heartbeat로 좀비 커넥션을 감지하고 끊긴 뒤 5초 후 자동 재연결되도록 했습니다.

- **구독 캐시 + 권한 유지:** 새 세션에서 등록해 둔 토픽이 자동 복원되도록 구독 캐시를 두고, 인증 토큰을 `connectHeaders` 와 각 `subscribe` 헤더에 모두 `Bearer` 형식으로 실어 백엔드 인터셉터 토큰 검증 흐름과 정합성을 맞췄습니다.
- **채널 이중화:** 혼잡도 전용 클라이언트를 채팅 클라이언트와 별도로 구성해 같은 SockJS 엔드포인트 위에 독립된 STOMP 세션을 띄우고 `/sub/session` 토픽 하나만 구독하도록 책임 범위를 좁혔습니다.
- **메시지 도메인 분리:** 혼잡도 데이터는 화면 갱신 콜백으로, 채팅 메시지는 채팅 렌더링 로직으로 책임을 분리했습니다.

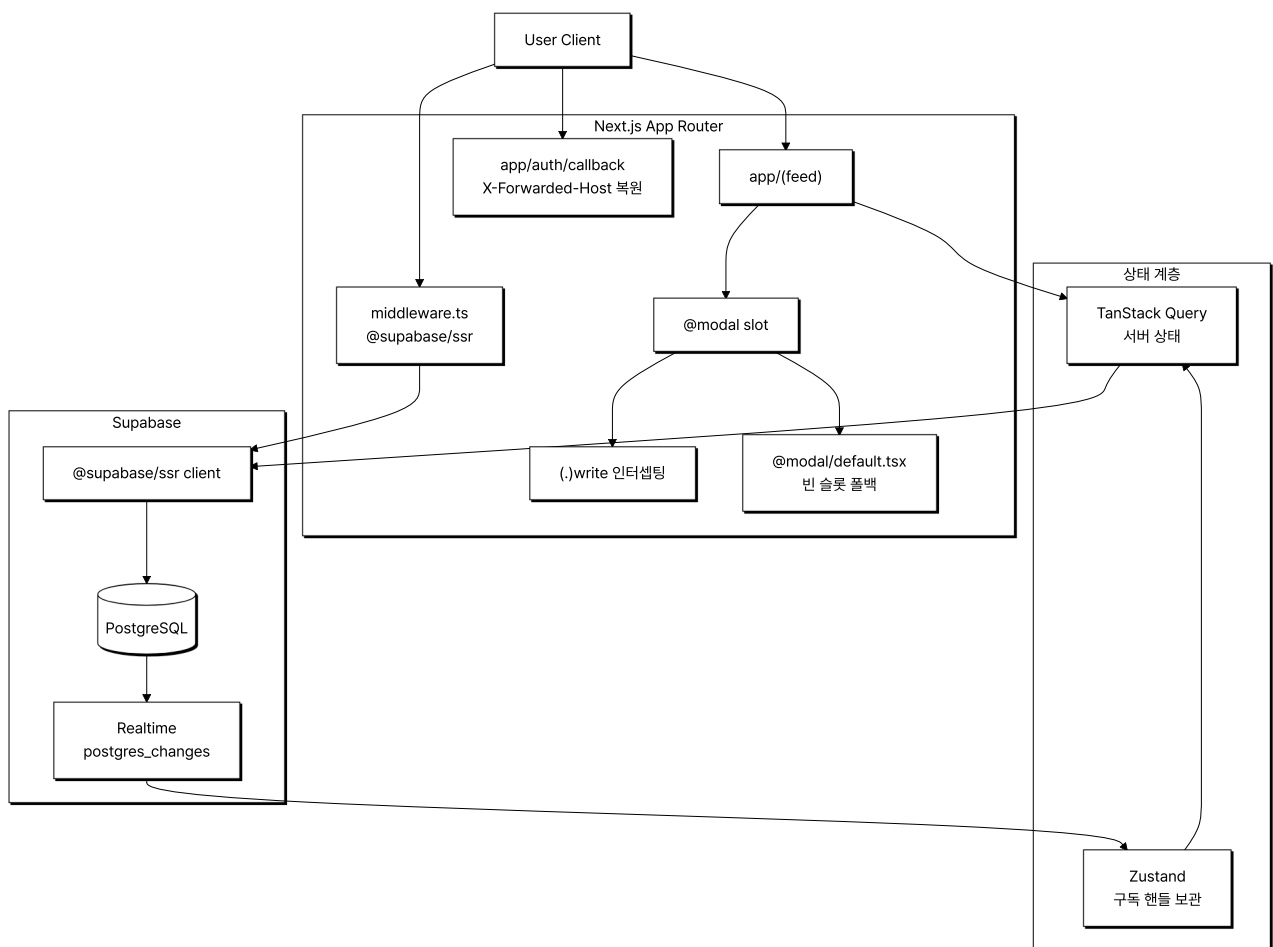
3. 결과

- **성과:** 망 단절 시 5초 안에 채팅·혼잡도 STOMP 세션이 자동 복구되고 구독·인증이 그대로 이어지며, 채팅과 혼잡도가 서로 다른 세션으로 흘러 한쪽 트래픽 폭증이 다른 쪽에 영향을 주지 않게 격리됐습니다.
- **배운 점:** 회복 정책은 공유하되 채널은 분리하는 패턴으로 두 트래픽의 운영 안정성을 동시에 확보했습니다.

[XAB] - 실시간 투표 및 소통 중심의 A/B 테스트 SNS

두 선택지에 대한 실시간 투표·댓글 토론을 반영하는 SNS로, 본인은 4인 FE 팀에 참여해 Next.js App Router 병렬·인터셉팅 라우트 모달·useInfiniteQuery 무한 스크롤·Supabase Realtime 구독 수명 관리·@supabase/ssr 미들웨어·OAuth 콜백 X-Forwarded-Host 복원을 담당했습니다.

전체적인 아키텍처



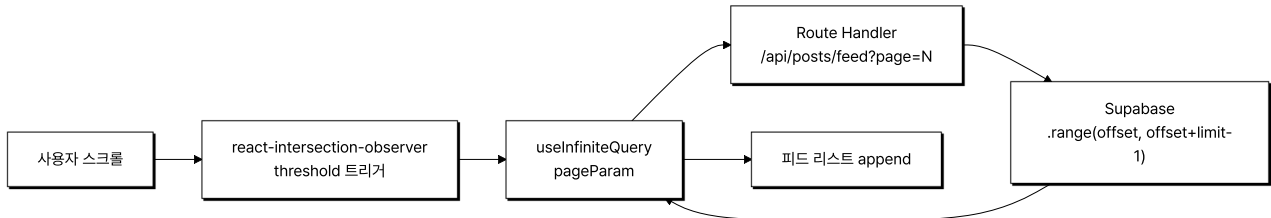
- **Architecture:** 병렬 라우트 (`@modal`) + 인터셉팅 라우트 (`(.)write`)로 모달·독립 페이지를 같은 URL로 처리하고, `@supabase/ssr` 미들웨어가 매 요청에서 세션을 검증하며, Realtime 구독 핸들을 Zustand에 보관해 라이프사이클을 통제합니다.

Case 1. useInfiniteQuery + Intersection Observer 무한 스크롤

1. 문제 원인

- 메인 피드 접속 시 게시물·투표 옵션·댓글 수·좋아요 정보를 한 번에 불러오는 구조에서 첫 화면 진입 응답 페이로드와 렌더링 비용이 누적되었습니다.
- 뷰포트 밖 게시물까지 동일 비중으로 페칭·렌더링하면 초기 화면 그려지기까지의 체감 지연이 발생했습니다.

2. 해결 과정



- useInfiniteQuery 페이지네이션:** TanStack Query의 `useInfiniteQuery` 로 페이지네이션 상태를 관리하고 `pageParam` 을 다음 페이지 요청에 전달.
- Intersection Observer 트리거:** `react-intersection-observer` 로 뷰포트 하단 트리거 요소 노출 시점을 감지해 다음 페이지를 fetch.
- Supabase range:** Route Handler가 `page` 쿼리 파라미터를 받아 `.range(offset, offset + limit - 1)` 로 페이지 단위 응답만 반환하도록 서버·클라이언트 분할 단위를 일치.

3. 결과

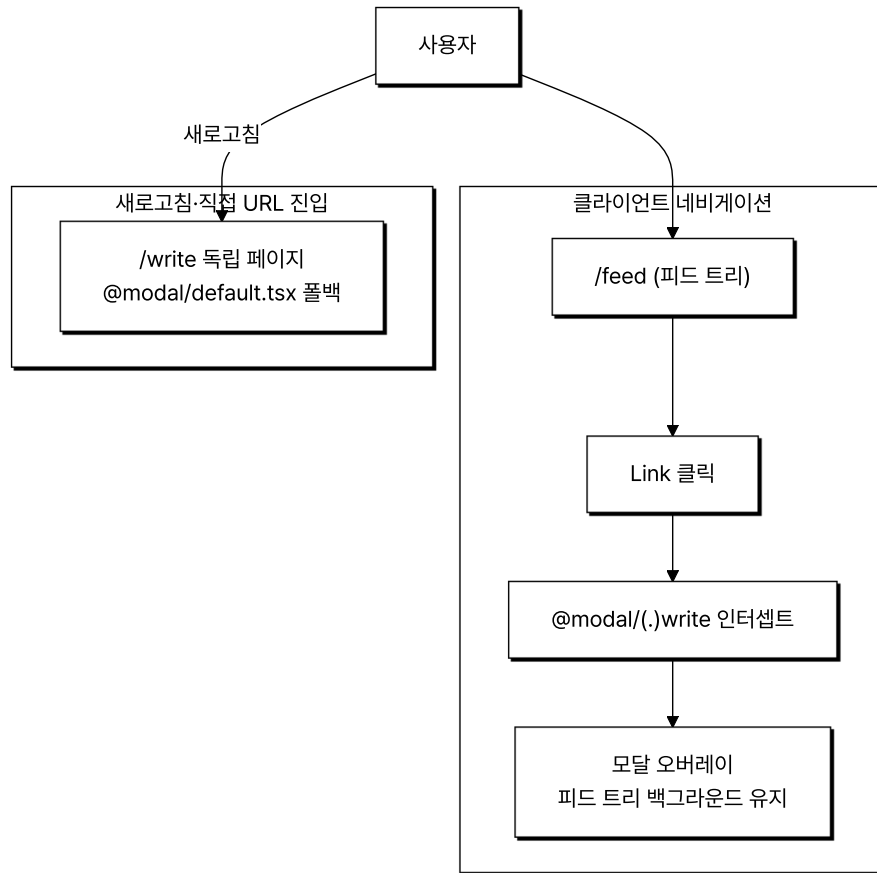
- 성과:** 초기 진입 데이터가 줄어 첫 화면 체감 지연이 완화되고, 스크롤 위치에 따라 점진적으로 데이터가 채워지는 무한 스크롤을 구현했습니다.
- 배운 점:** `useInfiniteQuery` + `Intersection Observer` + `Supabase .range()`를 일치시켜 초기 응답 페이로드를 페이지 단위로 줄였고 스크롤에 따라 점진적으로 채워졌습니다.

Case 2. 병렬 라우트 슬롯(@modal) + 인터셉팅 라우트((.)write)로 모달·독립 페이지를 같은 URL로

1. 문제 원인

- 피드에서 게시물 작성·상세 화면으로 이동할 때 전체 페이지가 다시 렌더되며 스크롤 위치·인터랙션이 초기화되었습니다.
- 기존 라우팅은 페이지 컨텍스트를 완전히 교체해 이전 화면 상태를 유지한 채 상호작용할 수 없는 한계가 있었습니다.

2. 해결 과정



- 병렬 라우트 슬롯: `app/@modal` 슬롯과 메인 라우트를 병렬 배치해 두 트리를 동시에 렌더링할 수 있는 구조를 만들었습니다.
- 인터셉팅 라우트: `@modal/(.)write` 인터셉팅으로 클라이언트 네비게이션 시 모달 오버레이가 렌더, 직접 URL 진입·새로고침 시 독립 페이지가 렌더되도록 분기.
- 빈 슬롯 폴백: `@modal/default.tsx` 에 빈 슬롯 폴백을 두어 슬롯이 활성화되지 않은 라우트에서도 트리가 흐트러지지 않게 동작.

3. 결과

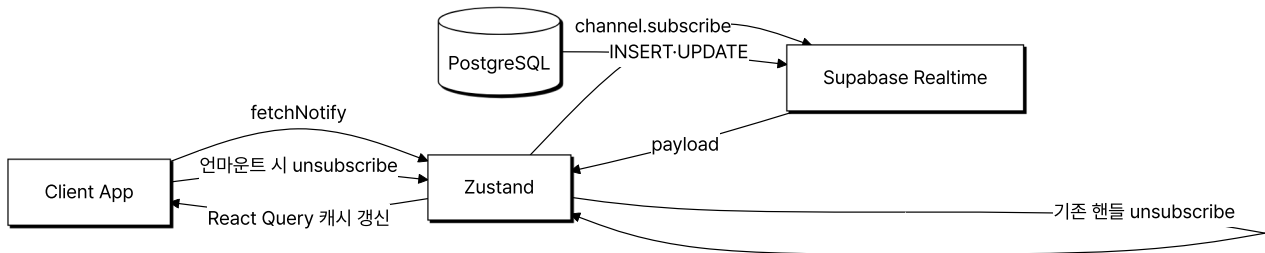
- 성과: 게시물 작성·상세 진입 시 피드 위치·스크롤 컨텍스트가 보존되어 탐색이 끊기지 않고, 모달·독립 페이지 진입을 같은 URL로 처리해 공유 링크·새로고침 시에도 일관된 화면을 갖췄습니다.
- 배운 점: `@modal` 슬롯 + `(.)write` 인터셉팅을 결합해 클라이언트 네비게이션은 모달 오버레이, 새로고침·직접 URL 진입은 독립 페이지로 분기되도록 같은 URL에서 두 진입 방식을 처리했습니다.

Case 3. Supabase Realtime postgres_changes + Zustand 구독 핸들 수명 관리

1. 문제 원인

- SNS 특성상 댓글·투표가 빈번하게 발생하지만 사용자가 새로고침을 누르기 전까지 최신 데이터를 확인할 수 없는 정적 환경이 소통을 저해했습니다.
- 주기적 폴링은 변화 없을 때도 호출이 반복되어 서버 부하·네트워크 비용을 누적.
- 구독 핸들을 관리하지 않으면 컴포넌트 마운트·언마운트·사용자 전환 시 중복 구독으로 인한 리소스 누수가 발생.

2. 해결 과정



- **postgres_changes 구독**: Supabase Realtime의 **postgres_changes** 이벤트로 DB 변경 사항을 WebSocket으로 수신.
- **TanStack Query 캐시 갱신**: 댓글 영역에서 새 댓글 도착 시 React Query 캐시를 갱신해 화면이 자동 반영되도록 연동.
- **구독 핸들 보관**: 알림 영역에서 Zustand 스토어가 구독 해제 핸들을 함께 보관해 재진입·사용자 전환 시 이전 구독을 정리한 뒤 재구독, 중복 구독 누수 방지.

3. 결과

- **성과**: 새로고침 없이 새 댓글·알림이 반영되는 라이브 환경을 갖췄고, 폴링 제거로 불필요한 요청을 줄였습니다.
- **배운 점**: Zustand 스토어에 postgres_changes 구독 해제 핸들을 함께 보관해 재진입·사용자 전환 시 이전 구독을 정리한 뒤 재구독하도록 만들어 중복 구독 누수가 사라졌습니다.

Case 4. @supabase/ssr 미들웨어 인증 + OAuth 콜백 X-Forwarded-Host 오리진 복원

1. 문제 원인

- 인증되지 않은 사용자의 보호 경로 접근을 차단하려면 서버 측 세션 검증이 필요했고, 클라이언트 단독 검증은 보호 라우트 fallback이 깜빡이는 결함이 있었습니다.
- 로드밸런서·리버스 프록시 환경에서 OAuth 콜백 URL을 만들 때 **request.url**의 host가 내부 호스트로 보여 잘못된 도메인으로 리다이렉트되는 사고가 발생할 수 있었습니다.
- 미들웨어 인증·OAuth 콜백 두 흐름이 분리되어 있으면 인증 사고가 어느 경로에서 발생했는지 추적이 어려웠습니다.

2. 해결 과정

두 흐름을 한 시스템 안에 함께 두었습니다.

- **인증 흐름**: 사용자 요청을 받은 **middleware.ts**가 **getUser**로 세션을 검증해 보호 라우트면 인증 분기, 미인증이면 로그인 리다이렉트로 보냅니다
- **OAuth 콜백 흐름**: OAuth provider 콜백이 **app/auth/callback**에 도착하면 X-Forwarded-Host 헤더로 원본 오리진을 복원해 올바른 도메인으로 리다이렉트합니다
- **@supabase/ssr 미들웨어**: **middleware.ts**에서 **@supabase/ssr** 기반으로 매 요청마다 서버 측 세션을 검증하고 미인증 사용자를 로그인 페이지로 리다이렉트.
- **getUser() 호출**: 미들웨어가 **supabase.auth.getUser()**로 토큰 유효성을 매 요청 검증해 만료 토큰 사용을 차단.
- **X-Forwarded-Host 우선 참조**: OAuth 콜백 라우트(**app/auth/callback**)에서 **request.headers.get('x-forwarded-host')**를 우선 참조해 원본 오리진을 복원, 로드밸런서 환경에서도 올바른 도메인으로 리다이렉트.
- **콜백 라우트 분리**: 콜백 라우트는 **[locale]**에 종속시키지 않아 OAuth provider 측 등록 리다이렉트 URL 관리 비용을 줄였습니다.

3. 결과

- **성과**: 미인증 사용자 보호 경로 접근이 서버 단계에서 차단되어 클라이언트 fallback 깜빡임이 사라지고, 로드밸런서 환경에서도 OAuth 콜백이 원본 도메인으로 정확히 돌아옵니다.

- **배운 점:** middleware.ts의 getUser()로 매 요청 세션을 검증하고 OAuth 콜백에서 x-forwarded-host를 우선 참조하도록 두어 보호 라우트 fallback 깜빡임과 로드밸런서 환경의 잘못된 리다이렉트가 함께 사라졌습니다.